

TÉCNICO EM PROGRAMAÇÃO DE JOGOS DIGITAIS

Lógica de Programação com GDScript

Godot 4.6+ • material de aula e de consulta

```
func _ready() -> void:  
    print("Olá, turma!")
```

PROFESSOR

Tarik Ponciano

ANTES DE COMEÇAR

Como usar este material

Este deck é acumulativo: cada aula ganha uma nova seção. Use-o na aula expositiva e depois como consulta — os códigos estão prontos para digitar na Godot.

```
var vida := 100
```

Blocos de código

Fundo escuro, com as mesmas cores do editor da Godot. Quando existe, o nome do arquivo aparece no topo.

ATIVIDADE 3

Atividades

O briefing (fundo escuro) conta a missão; o slide laranja traz desafios para resolver, não para copiar.

ATENÇÃO

Caixas de atenção

Armadilhas comuns: erros que a Godot aponta ou resultados que surpreendem.

AULA 01

Roteiro da aula

01 Preparação do ambiente

Godot 4.6+, projeto e Terminal da turma

02 Olá, mundo!

Primeiro script, Saída e Terminal

03 Funções nativas

_ready, _process e o ciclo de vida

04 Declaração de variáveis

var, const, tipagem e annotations

05 Tipos primitivos

bool, int, float e String

06 Operações com variáveis

Aritméticas, lógicas e de texto

07 Manipulação de tipos

Descobrir, converter e validar

08 Entrada de dados

Terminal, Inspetor e teclado

AULA 01 • SEÇÃO 01

Preparação do ambiente

Instalar a Godot, criar o projeto do curso e organizar cada exercício como cena + script.

01

Instalando a Godot 4.6+ no Mac

1 Baixe

Em godotengine.org > Download > macOS, escolha a versão **Standard**. A versão .NET é para C# e não será usada.

2 Instale

Abra o .zip baixado e arraste o **Godot.app** para a pasta **Aplicativos** (Applications). Funciona em Macs com chip Apple ou Intel.

3 Abra

Abra pelo Launchpad ou pela pasta Aplicativos. Se o macOS avisar que é um app da internet, clique em **Abrir**.

4 Confira a versão

Toda a turma deve usar 4.6 ou superior. A versão aparece no Gerenciador de Projetos e no rodapé do editor.



Atalhos no Mac: as teclas F1–F12 controlam brilho e volume, então a Godot usa a tecla **Cmd**. Exemplo: **Cmd+R** roda a cena aberta.

Criando o projeto do curso

- 1 No Gerenciador de Projetos, clique em **Criar** (Create).
- 2 Nome do projeto: **logica-gdscript**.
- 3 Caminho: uma pasta vazia, só para o curso.
- 4 Renderizador: **Compatibilidade** (Compatibility).
- 5 Controle de versão: **Git** — a Godot cria o .gitignore e o .gitattributes.
- 6 Confirme para abrir o editor.

Atalho: projeto-modelo

Recebeu o projeto-modelo da turma? Ele já vem assim, com o Terminal instalado. Descompacte e use **Importar** em vez de Criar.

Editor em português?

No Mac: menu **Godot**, no topo da tela > Configurações do Editor > Interface > Editor > Idioma do Editor. Neste material, os menus vêm em português, com o nome em inglês entre parênteses.

Instalando o Terminal da turma

O **terminal.gd** é um console dentro da janela do jogo: escreve como o `print()` e pergunta coisas ao usuário. Instale uma vez por projeto.

- 1 No painel Sistema de Arquivos (FileSystem), crie a pasta **terminal/**.
- 2 Arraste o **terminal.gd** do Finder para dentro dessa pasta.
- 3 **Projeto > Configurações do Projeto > aba Globais > Autoload.**
- 4 Caminho: **res://terminal/terminal.gd**. Nome do nó: **Terminal**. Clique em **Adicionar**.

O que é um Autoload?

Um script que a Godot carrega sozinha antes de qualquer cena. Todos os scripts do projeto passam a enxergá-lo pelo nome: **Terminal**.

```
teste rápido  
  
func _ready() -> void:  
    Terminal.escrever("Terminal pronto!")
```

No projeto-modelo da turma, o Terminal já vem instalado. Estes passos servem para projetos novos.

Organização do repositório

estrutura sugerida

```
logica-gdscript/  
├─ project.godot  
├─ terminal/  
│   └─ terminal.gd  
├─ aula01/  
│   ├── atv01_hello_world.tscn  
│   ├── atv01_hello_world.gd  
│   └─ ...  
├─ aula02/  
└─ lista01/  
    ├── q01_boas_vindas.tscn  
    └─ q01_boas_vindas.gd
```

Uma cena por exercício

Cada exercício roda sozinho com **Cmd+R**.

Mesmo nome, mesma pasta

A cena **atv03_operacoes.tscn** usa o script **atv03_operacoes.gd**.

Nomes em snake_case

Sem acentos nem espaços. Prefixos: **atv** (aula) e **q** (lista).

terminal/ é da turma toda

Guarda o Terminal usado por todas as cenas. Não altere.

.godot/ é cache

A engine recria sozinha; o .gitignore a deixa fora do Git.

Criando a cena de um exercício

- 1 **Cena > Nova Cena** (Scene > New Scene).
- 2 Na aba Cena, escolha **Outro Nó** (Other Node) e busque por **Node**.
- 3 Renomeie o nó raiz com duplo clique. Exemplo: **HelloWorld**.
- 4 **Cmd+S** e salve como **aula01/atv01_hello_world.tscn**.
- 5 Clique com dois dedos no nó (clique secundário) > **Anexar Script** (Attach Script). Confira: mesmo nome, mesma pasta.
- 6 Escreva o código e rode com **Cmd+R**.

Atalhos no Mac

Cmd+R

Roda a cena aberta. Use nos exercícios.

Cmd+B

Roda o projeto (a cena principal).

Cmd+.

Para a execução (Cmd + ponto).

Cmd+S

Salva. Ao rodar, a Godot também salva tudo.

No Windows: F6, F5, F8 e Ctrl+S.

Primeira cena

PASSOS

- 1 Abra o **projeto-modelo** da turma: Gerenciador de Projetos > **Importar** > project.godot.
- 2 Crie a cena **aula01/atv01_ola.tscn**, com um Node como raiz, e anexe um script com o mesmo nome.
- 3 No `_ready()`, faça a janela do jogo mostrar: **Olá, eu sou seu nome!**
- 4 Rode com **Cmd+R** e confira a mensagem na janela e no painel Saída.

PRONTO QUANDO

- ✓ A mensagem aparece na janela e na Saída.
- ✓ `atv01_ola.tscn` e `atv01_ola.gd` estão lado a lado em `aula01/`.

Sem o projeto-modelo? Siga os slides “Criando o projeto do curso” e “Instalando o Terminal da turma”.

A ROTINA DE TODO EXERCÍCIO

- 1 Nova cena, raiz **Node**



- 2 **Cmd+S** na pasta da aula



- 3 Anexar script, mesmo nome



- 4 Escrever no `_ready()`



- 5 **Cmd+R** e conferir a Saída

AULA 01 • SEÇÃO 02

Olá, mundo!

O primeiro script, o painel Saída (Output), a família print e o texto formatado.

02

Anatomia de um script

```
modelo da Godot, comentários traduzidos

extends Node

# Chamada quando o nó entra na cena.
func _ready() -> void:
    pass # Substitua pelo corpo da função.

# Chamada a cada quadro (frame).
func _process(delta: float) -> void:
    pass
```

extends Node	O script estende um tipo de nó e herda tudo o que ele sabe fazer.
# texto	Comentário: a Godot ignora. Serve para pessoas.
func nome():	Declara uma função. Os dois-pontos abrem o bloco.
-> void	Tipo de retorno: void = a função não devolve valor.
Tab	A indentação define o que está dentro da função. Não há chaves { }.
pass	“Não faça nada”: marca um bloco ainda vazio.

Olá, mundo! no painel Saída

```
atv01_hello_world.gd  
  
extends Node  
  
func _ready() -> void:  
    print("Olá, mundo!")
```

SAÍDA (OUTPUT)

```
Godot Engine v4.6.stable.official ...  
Olá, mundo!
```

- 1 Salve o script (**Cmd+S**).
- 2 Pressione **Cmd+R** para rodar a cena atual.
- 3 Olhe o painel **Saída** (Output), na parte de baixo do editor.



Atenção: o `print()` escreve só na Saída; a janela do jogo fica vazia. Para escrever na janela, use o Terminal da turma (próximo slide).

A família print

FUNÇÃO	EXEMPLO	SAÍDA	O QUE FAZ
print	print("Vida:", 10)	Vida:10	Junta os valores sem espaço
prints	prints("Vida:", 10)	Vida: 10	Separa os valores com espaço
printt	printt("Vida:", 10)	Vida: 10	Separa com tabulação (colunas)
print_rich	print_rich("[b]Oi[/b]")	Oi	Aceita BBCode: [b], [i], [color=red]
push_warning	push_warning("Cuidado")	Cuidado	Aviso em amarelo, também no Depurador
push_error	push_error("Falhou")	Falhou	Erro em vermelho, também no Depurador



Dica: print() não coloca espaço entre os valores. Escreva o espaço dentro do texto – **print("Vida: ", 10)** – ou use texto formatado (próximo slide).

Texto formatado com %

<code>"Vida: %d/%d"</code>	<code>%</code>	<code>[vida, vida_max]</code>
MODELO COM MARCADORES	OPERADOR	VALORES, NA ORDEM

```
var nome: String = "Lia"
var vida: int = 80
var vida_max: int = 100

print("Vida: %d/%d" % [vida, vida_max])
print("Olá, %s!" % nome)
print("%s tem %d de vida" % [nome, vida])
```

SAÍDA (OUTPUT)

```
Vida: 80/100
Olá, Lia!
Lia tem 80 de vida
```

- ✓ Cada marcador é trocado por um valor, **na ordem**.
- ✓ Um valor: direto. Vários: dentro de `[]`.
- ✓ Vale em qualquer texto, inclusive no `Terminal.escrever()`.

Os marcadores mais usados

MARCADOR	EXEMPLO	RESULTADO	PARA QUÊ
%s	"Olá, %s" % "Lia"	Olá, Lia	Qualquer valor, convertido em texto
%d	"Nível %d" % 7	Nível 7	Inteiro (um float perde os decimais)
%.2f	"R\$ %.2f" % 9.5	R\$ 9.50	Decimal com 2 casas (arredonda)
%03d	"Fase %03d" % 7	Fase 007	Inteiro com zeros à esquerda
%5d	"[%5d]" % 42	[42]	Ocupa 5 espaços, alinhado à direita
%-8s	"[%-8s]" % "Lia"	[Lia]	Ocupa 8 espaços, alinhado à esquerda
%%	"%d%%" % 50	50%	Escreve o próprio símbolo %



Atenção: o número de marcadores precisa ser igual ao número de valores. Faltou ou sobrou valor? A Godot mostra um erro ao rodar.

Terminal.escrever: o print na janela do jogo

```
extends Node

var vida: int = 80

func _ready() -> void:
    print("Vida: ", vida)
    Terminal.escrever("Vida: ", vida)
    Terminal.escrever("[color=yellow]Cuidado![/color]")
```

- ✓ Funciona como o print(): vários valores separados por vírgula, juntos **sem espaço**.
- ✓ Aceita cores e negrito, como o print_rich().
- ✓ Aparece na janela do jogo e também na Saída.

JANELA DO JOGO

Vida: 80
Cuidado!

SAÍDA (OUTPUT)

Vida: 80
Vida: 80
Cuidado!

Para limpar a janela: **Terminal.limpar()**.

ATIVIDADE 2 • BRIEFING

Missão: a tela de apresentação

Tempo: 20 min

Formato: em dupla

Entrega: aula01/atv02_apresentacao.tscn

O jogo da turma vai abrir com a apresentação de cada jogador. O designer mandou só o resultado final. Cabe à dupla descobrir qual ferramenta gera cada linha.

REGRAS DA MISSÃO

- › Cada linha usa uma ferramenta diferente: print, prints, printt, print_rich e texto formatado com %.
- › Nome, idade, cidade, fase e vida ficam em variáveis – nada escrito direto no texto.
- › Trocou os valores das variáveis? A saída tem que continuar certa.

OBJETIVO

Reproduzir a Saída-alvo caractere por caractere e provar que ela se ajusta a qualquer jogador.

VOCÊ VAI USAR

print • prints • printt

print_rich

%s • %d • %03d • %%

ATIVIDADE 2 • MISSÃO

A tela de apresentação

DESAFIOS

- 1 Detetive:** para cada linha da Saída-alvo, anote no caderno qual ferramenta a produz.
- 2** No topo do script, declare nome, idade, cidade, fase, vida e jogo, com os valores da Saída-alvo.
- 3** No `_ready()`, escreva uma linha de código para cada linha da Saída-alvo e confira caractere por caractere.
- 4 Teste:** mude fase para 12 e vida para 5. A linha 4 virou “Fase 012 | Vida 5%”?
- 5 Bônus:** mostre a linha 4 também na janela, com `Terminal.escrever()`.

SAÍDA-ALVO

```
Olá! Eu sou Ana.  
Ana 16 Recife  
Ana→16→Recife  
Fase 007 | Vida 80%  
Jogo favorito: Celeste  
→ = tabulação • a última palavra é colorida
```

PRONTO QUANDO

- ✓ A Saída é idêntica à alvo.
- ✓ Trocar os valores não quebra nada.

PENSE E RESPONDA Por que a linha 4 fica difícil só com `print(a, b, c)`? O que o `%` resolve?

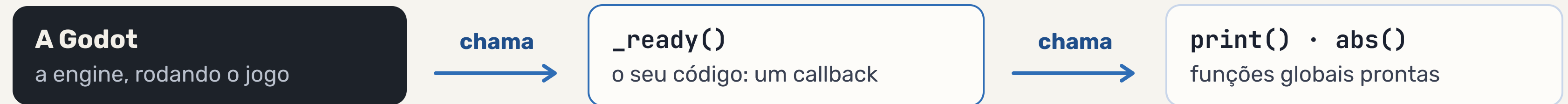
AULA 01 • SEÇÃO 03

Funções nativas da Godot

_ready, _process e companhia: funções que a própria engine chama no momento certo.

03

Callbacks e funções globais



A GODOT CHAMA

Callbacks

Você escreve o corpo; a engine decide quando executar. Começam com `_` (sublinhado).

```
func _ready() -> void:
    print("A Godot me chamou!")
```

Você não precisa chamar `_ready()` no seu código.

VOCÊ CHAMA

Funções globais

Prontas para usar em qualquer script, na hora que você quiser.

```
print(abs(-5))           # 5
print(max(3, 8))         # 8
print(randi_range(1, 6)) # de 1 a 6
```

Várias delas aparecem nas próximas seções.

ATIVIDADE 3 • BRIEFING

Missão: o maestro da Godot

Tempo: 20 min

Formato: em dupla

Entrega: aula01/atv03_ciclo.tscn

A Godot funciona como um maestro: ela chama cada função do seu script na hora certa. A dupla vai descobrir essa ordem como cientistas — por experimento, sem olhar a resposta.

REGRAS DA MISSÃO

- › Hipótese antes do experimento: escrevam a ordem no caderno antes de rodar.
- › As funções devem aparecer no arquivo fora de ordem, de propósito.
- › O `_process` não pode inundar a Saída.

OBJETIVO

Descobrir em que ordem a Godot chama `_init`, `_enter_tree`, `_ready`, `_process` e `_exit_tree` — e provar com o seu próprio código.

VOCÊ VAI USAR

`print`

`callbacks`

`set_process(false)`

O maestro da Godot

DESAFIOS

- 1 **Hipótese:** na ficha ao lado, escrevam em que ordem a Godot chama `_init`, `_enter_tree`, `_ready`, `_process` e `_exit_tree`.
- 2 **Experimento:** criem `atv03_ciclo.tscn` com as 5 funções, fora de ordem no arquivo. Cada uma escreve o próprio nome, ex.: `print("_ready")`.
- 3 **Problema:** o `_process` repete 60 vezes por segundo. Façam ele escrever uma vez só. Dica: `set_process(false)`.
- 4 **Mistério:** o `_exit_tree` não apareceu? Parem a cena com `Cmd+. e`, depois, fechando a janela. Em qual caso ele aparece?
- 5 Comparem com a hipótese. Quantas posições vocês acertaram?

FICHA DA HIPÓTESE

1°

2°

3°

4°

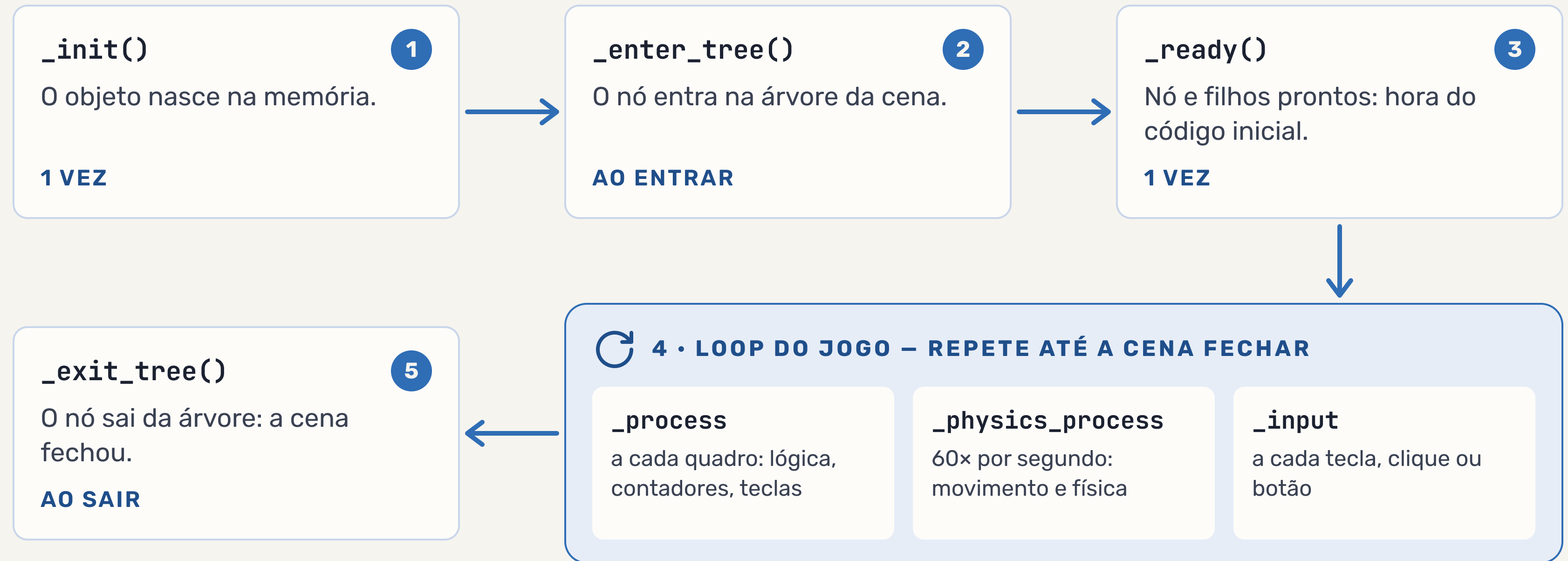
5°

PRONTO QUANDO

✓ Cada função aparece uma vez só.

PENSE E RESPONDA A ordem das funções no arquivo mudou a ordem na Saída? Então quem decide a ordem?

O ciclo de vida de um nó



A ordem é sempre esta – não importa em que ordem as funções aparecem no arquivo.

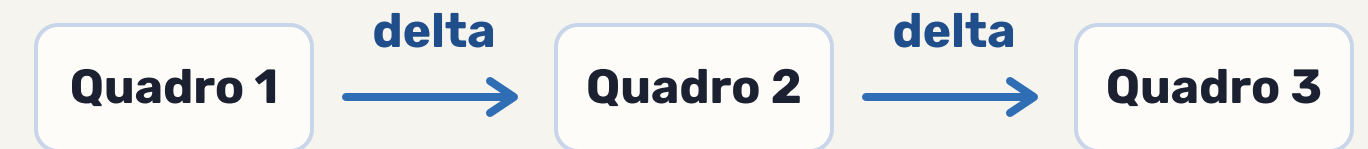
_process e o delta

```
extends Node

var quadros: int = 0
var tempo_total: float = 0.0

func _process(delta: float) -> void:
    quadros += 1
    tempo_total += delta
    print("Quadro ", quadros, " | tempo: ", tempo_total)
```

Não vai usar o delta? Escreva **_delta** e a Godot deixa de avisar que o parâmetro não é usado.



A 60 quadros por segundo, cada delta vale cerca de **0,016 s** ($1 \div 60$).

delta é o tempo, em segundos, desde o quadro anterior. Somando os deltas, você mede quanto tempo passou.



Atenção: print() no _process() escreve cerca de 60 linhas por segundo. Observe e pare com **Cmd+.** (Cmd + ponto).

AULA 01 • SEÇÃO 04

Declaração de variáveis

Guardar valores com nome: var, const, tipagem e as annotations que ligam o código ao Inspetor.

04

O que é uma variável?

```
var   vida  : int = 100
```

PALAVRA-CHAVE NOME TIPO VALOR INICIAL

Uma **variável** é um espaço na memória, com um nome, que guarda um valor. Esse valor pode mudar enquanto o programa roda.

No jogo: a variável vida guarda 100 no começo; depois de um golpe, passa a guardar 75. O nome continua o mesmo.

Três formas de declarar

Dinâmica

```
var pontos = 0
```

Sem tipo: aceita qualquer valor depois. Flexível, mas os erros só aparecem com o jogo rodando.

```
pontos = "dez" aceito
```

Tipada

```
var pontos: int = 0
```

O tipo fica fixo. A Godot aponta erros antes de rodar e o autocompletar fica mais esperto.

```
pontos = "dez" erro no editor
```

Inferida

```
var pontos := 0
```

O tipo vem do valor inicial (0 é int). Mais curta e tão segura quanto a tipada.

```
pontos = "dez" erro no editor
```



Combinado da turma: sempre declarar com tipo – **: Tipo** ou **:=**. Assim os erros aparecem antes de o jogo rodar.

Constantes e nomes

```
const VIDA_MAXIMA: int = 100
const GRAVIDADE := 9.8
const NOME_DO_JOGO := "Dungeon Lógica"

func _ready() -> void:
    VIDA_MAXIMA = 200 # ERRO: constante não muda
```

const guarda um valor fixo: definido uma vez, nunca alterado. Use para regras do jogo que não mudam.

PARA	PADRÃO	EXEMPLO
Variáveis e funções	snake_case	vida_atual
Constantes	CAIXA_ALTA	VIDA_MAXIMA
Nós e classes	PascalCase	HelloWorld
Arquivos	snake_case	hello_world.gd

Nomes inválidos

- ~~2vidas~~ começa com número
- ~~vida-max~~ o hífen é lido como subtração
- ~~func~~ palavra reservada da linguagem

Escopo: onde a variável existe

```
extends Node

var pontos: int = 0 # membro: vale no script todo

func _ready() -> void:
    var bonus: int = 50 # local: só existe aqui
    pontos += bonus
    print(pontos) # 50

func _exit_tree() -> void:
    print(pontos) # 50: o membro existe aqui
    print(bonus) # ERRO: bonus é local
```

Variável membro

Declarada fora das funções, no topo do script. Existe enquanto o nó existir e todas as funções a enxergam.

Variável local

Declarada dentro de uma função. Nasce quando a função começa e some quando ela termina.

Regra prática: no topo, o que precisa durar; dentro da função, o que é temporário.

@export: variáveis no Inspetor

```
extends Node

@export var nome: String = "Herói"
@export var vida: int = 100
@export var velocidade: float = 3.5
@export var tem_escudo: bool = false

func _ready() -> void:
    prints(nome, vida, velocidade, tem_escudo)
```

SAÍDA (OUTPUT)

Herói 100 3.5 false

INSPETOR (INSPECTOR)

Nome	Herói
Vida	100
Velocidade	3.5
Tem Escudo	☐ On

Selecione o nó raiz na aba Cena (Scene) para ver as variáveis exportadas.

Mudou no Inspetor? O novo valor fica salvo na cena (.tscn) e **vale mais que o do código**. A seta circular ao lado do campo desfaz.

Annotations mais usadas

ANNOTATION	EXEMPLO	O QUE FAZ
@export	@export var vida: int = 100	Mostra a variável no Inspetor
@export_range	@export_range(0, 100) var volume: int = 50	Número com limites (vira um slider)
@export_multiline	@export_multiline var fala: String	Caixa de texto com várias linhas
@export_enum	@export_enum("Mago", "Elfo") var classe: String	Lista de opções para escolher
@export_group	@export_group("Atributos")	Agrupa as exportações seguintes
@onready	@onready var rotulo: Label = \$Label	Pega um nó filho quando a cena fica pronta
@tool	@tool # na 1ª linha do script	Faz o script rodar dentro do editor

ATIVIDADE 4 • BRIEFING

Missão: à prova de designer

Tempo: 25 min

Formato: em dupla

Entrega: aula01/atv04_ficha.tscn

A designer quer ajustar os personagens sozinha, pelo Inspetor. Da última vez, ela pôs nível 999 e escreveu “Magoo” na classe. O jogo quebrou.

DADO	O PEDIDO DA DESIGNER
nome	“Qualquer nome.”
classe	“Só Guerreiro, Mago ou Arqueiro.”
nível	“De 1 a 50.”
vida	“Um número inteiro.”
velocidade	“Pode ter casas decimais, de 0 a 10.”
eh_chefe	“Sim ou não.”
nome do jogo	“Dungeon Lógica. Nunca muda.”

OBJETIVO

Uma ficha de personagem que a designer edita pelo Inspetor — sem conseguir quebrar.

REGRAS DA MISSÃO

- A dupla decide o tipo e a annotation de cada dado — e justifica.
- Nenhum valor inválido pode ser digitado no Inspetor.
- O que nunca muda não aparece no Inspetor.

À prova de designer

DESAFIOS

- 1 Para cada dado do pedido, decidam o tipo, var ou const, e a annotation. Montem a tabela no caderno.
- 2 Implementem a ficha e organizem o Inspetor em dois grupos.
- 3 No `_ready()`, mostrem a ficha, um dado por linha, com texto formatado.
- 4 **Teste de estresse:** troquem de computador com outra dupla e tentem quebrar a ficha dela pelo Inspetor.

MODELO DA TABELA

Dado	Tipo	var/const	Annotation
nivel	int	var	@export_range
classe	?	?	?

PRONTO QUANDO

- ✓ A outra dupla não conseguiu pôr um valor inválido.
- ✓ O nome do jogo não aparece no Inspetor.

PENSE E RESPONDA Qual dado virou const? Por que ele não aparece no Inspetor?

AULA 01 • SEÇÃO 05

Tipos primitivos

Os tipos básicos de todo valor em GDScript: bool, int, float e String.

05

Os quatro tipos primitivos

bool Verdadeiro ou falso

true false

```
var vivo: bool = true
```

int Números inteiros

42 -7 0

```
var moedas: int = 250
```

float Números com parte decimal

3.14 -0.5 2.0

```
var gravidade: float = 9.8
```

String Texto, entre aspas

"Olá" 'A' ""

```
var nome: String = "Lia"
```

int e float em detalhe

int inteiro de 64 bits

Vai de cerca de $-9,2$ a $+9,2$ quintilhões. Nunca tem parte decimal.

```
var moedas := 1_000_000 # 1 milhão
var cor := 0xFF          # hexa: 255
var bits := 0b1010       # binário: 10
var zerado: int           # vale 0
```

float real de 64 bits

No código, o separador decimal é sempre o **ponto**: 3.14, nunca 3,14.

```
var pi := 3.14159
var dois := 2.0          # ainda é float
var mil := 1e3            # 1000.0
var zerado: float         # vale 0.0
```

 **Atenção:** floats são aproximações. **$0.1 + 0.2 == 0.3$** dá false! Para comparar floats, use **`is_equal_approx(a, b)`**.

String em detalhe

```
var nome := "Lia"  
var fala := 'Ela disse "oi" '  
var aspas := "Ele disse \"oi\""  
var linhas := "Linha 1\nLinha 2"  
var colunas := "A\tB"  
var longo := ""Texto com  
várias linhas""
```

\n	nova linha
----	------------

\t	tabulação
----	-----------

\"	aspas dentro de aspas
----	-----------------------

\\	uma barra invertida
----	---------------------

Aspas simples ou duplas: tanto faz. Use uma para poder escrever a outra dentro do texto.

Tudo o que o jogador digita chega como String – até “42”. Para fazer conta, é preciso converter (Seção 07).

Além dos primitivos: tipos da engine

A Godot traz tipos prontos para jogos. Não são o foco desta aula, mas aparecem em todo lugar — veremos cada um nas próximas aulas.

Vector2

Posição ou direção 2D (x, y)

```
Vector2(10, 20)
```

Vector2i

Vector2 só com inteiros

```
Vector2i(3, 4)
```

Color

Cor RGBA, de 0.0 a 1.0

```
Color(1, 0, 0)
```

Array

Lista ordenada de valores

```
[10, 20, 30]
```

Dictionary

Pares chave → valor

```
{"vida": 100}
```

null

Ausência de valor

```
var alvo = null
```

ATIVIDADE 5 • BRIEFING

Missão: a alfândega de dados

Tempo: 15 min

Formato: equipes de 3

Entrega: aula01/atv05_tipos.tscn

Todo valor que entra no jogo passa pela alfândega de tipos. Um valor mal classificado vira bug: $42 + 1$ dá 43, mas “42” + 1 nem roda.

REGRAS DA MISSÃO

- › Rodada de apostas: a equipe anota o tipo de cada valor antes de rodar qualquer código.
- › 1 ponto por acerto. A prova oficial é `type_string(typeof(valor))`.
- › Vale discutir; não vale testar antes de apostar.

OBJETIVO

Classificar 10 valores sem rodar nada — e depois provar cada resposta com código.

VOCÊ VAI USAR

`bool • int • float • String`

`null`

`typeof • type_string`

ATIVIDADE 5 • MISSÃO

A alfândega de dados

42	42.0	"42"	true	"true"
-0.5	1e3	0xFF	'A'	null

DESAFIOS

- 1 **Apostem:** anotem o tipo de cada valor acima.
- 2 **Provem:** para cada valor, rodem `print(type_string(typeof(valor)))` e confirmam a aposta.
- 3 **Contra-ataque:** inventem 3 valores “pegadinha” e desafiem a equipe vizinha.

PLACAR DA EQUIPE

Acertos na aposta _____ / 10

Pegadinhas que enganaram _____ / 3

PENSE E RESPONDA

Por que o computador trata “42” e 42 como coisas diferentes? Dê um exemplo de bug que isso causaria num jogo.

AULA 01 • SEÇÃO 06

Operações com variáveis

Aritmética, comparação, lógica e texto — e as armadilhas de cada uma.

06

Operadores aritméticos

SÍMBOLO	OPERAÇÃO	EXEMPLO	RESULTADO
+	Soma	7 + 2	9
-	Subtração	7 - 2	5
*	Multiplicação	7 * 2	14
/	Divisão	7.0 / 2	3.5
%	Resto da divisão	7 % 2	1
**	Potência	2 ** 3	8

0 resto (%) no dia a dia

`n % 2`
0 se n é par, 1 se é ímpar

`125 % 60`
5 – os segundos que sobram depois de 2 minutos

`(turno + 1) % 4`
gira entre 0, 1, 2 e 3 – a vez de cada um dos 4 jogadores

Cuidado com a divisão inteira

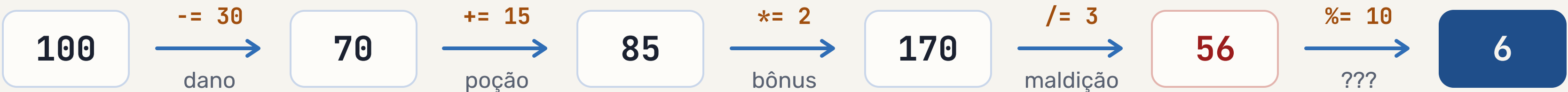
EXPRESSÃO	RESULTADO	POR QUÊ
<code>(7 + 8) / 2</code>	7	int ÷ int = int: a parte decimal é descartada
<code>(7 + 8) / 2.0</code>	7.5	se um dos lados é float, o resultado é float
<code>float(7 + 8) / 2</code>	7.5	converter antes de dividir também resolve
<code>-7 % 3</code>	-1	o resto segue o sinal do número dividido
<code>posmod(-7, 3)</code>	2	resto sempre positivo
<code>fmod(7.5, 2)</code>	1.5	% só aceita int; para float, use fmod()



Atenção: `var media: float = (7 + 8) / 2` guarda 7.0, não 7.5 – a divisão inteira acontece antes de o valor virar float. O editor mostra um aviso amarelo.

Atribuição composta

A vida de um herói ao longo de um combate, começando em **var vida: int = 100**:



Por que **56** e não 56.67? Como vida é int, 170 / 3 é uma divisão inteira.

ATALHO

É O MESMO QUE

a += b

a = a + b

a -= b

a = a - b

a *= b

a = a * b

a /= b

a = a / b

Também existem **%=** e ****=**.



Não existe ++ nem -- em GDScript. Em vez de contador++, escreva **contador += 1**.

Comparação e lógica

Comparação: o resultado é um bool

<code>==</code>	igual	<code>5 == 5 → true</code>
<code>!=</code>	diferente	<code>5 != 3 → true</code>
<code><</code>	menor	<code>2 < 1 → false</code>
<code>></code>	maior	<code>2 > 1 → true</code>
<code><=</code>	menor ou igual	<code>3 <= 3 → true</code>
<code>>=</code>	maior ou igual	<code>4 >= 9 → false</code>

Lógica: combina bools

<code>A</code>	<code>B</code>	<code>A and B</code>	<code>A or B</code>
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

not inverte: `not true → false`. Também valem `&&`, `||` e `!`.

```
var pode_atacar := energia >= 10 and not tonto
```



Não confunda: `=` guarda um valor na variável; `==` pergunta se dois valores são iguais.

Ordem das operações

DA MAIOR PARA A MENOR PRIORIDADE

- | | | |
|---|-----------------|-------------------------------|
| 1 | () | parênteses |
| 2 | ** | potência |
| 3 | -x | sinal negativo |
| 4 | * / % | multiplicação, divisão, resto |
| 5 | + - | soma e subtração |
| 6 | == != < > <= >= | comparações |
| 7 | not | negação lógica |
| 8 | and | e lógico |
| 9 | or | ou lógico |

```
print(2 + 3 * 4)           # 14
print((2 + 3) * 4)         # 20
print(10 - 4 - 3)          # 3
print(2 * 3 ** 2)          # 18
print(1 < 2 and 5 > 9)     # false
```

Mesmo nível? A conta segue da esquerda para a direita: 10 - 4 - 3 é (10 - 4) - 3.



Na dúvida, use parênteses. Eles garantem a ordem e deixam a intenção clara para quem lê.

Funções matemáticas úteis

FUNÇÃO	EXEMPLO	RESULTADO	USO COMUM
abs(x)	abs(-7)	7	valor sem sinal (distância)
min(a, b) • max(a, b)	max(1, 3 - 8)	1	garantir um dano mínimo
clamp(x, mín, máx)	clamp(120, 0, 100)	100	vida nunca passa do limite
sqrt(x)	sqrt(16)	4.0	raiz quadrada (distâncias)
pow(base, exp)	pow(2, 10)	1024.0	potência que devolve float
randi_range(a, b)	randi_range(1, 6)	1 a 6	sortear, como rolar um dado



Dica: existem versões tipadas – **clampi, clampf, mini, maxf...** O i ou o f no final garante que o resultado é int ou float.

Operações com texto

OPERAÇÃO	EXEMPLO	RESULTADO
Juntar (+)	"Game" + " Over"	"Game Over"
Repetir	"=".repeat(10)	"=========="
Formatar (%)	"Vida: %d" % 80	"Vida: 80"
Vários valores	"%s tem %d anos" % ["Lia", 16]	"Lia tem 16 anos"
Casas decimais	"%.2f" % 3.14159	"3.14"
Contém? (in)	"ouro" in "tesouro"	true
Tamanho	"Godot".length()	5
Maiúsculas	"godot".to_upper()	"GODOT"

MARCADORES DO % **%s** texto **%d** inteiro **%f** decimal **%.2f** decimal com 2 casas **%%** o próprio símbolo %

ATIVIDADE 6 • BRIEFING

Missão: balanceamento de combate

Tempo: 25 min

Formato: em dupla

Entrega: aula01/atv06_dano.tscn

O game designer escreveu as regras de dano num guardanapo, na hora do almoço. A dupla vai transformar essas regras em código — e provar, com casos de teste, que está tudo certo.

REGRAS DA MISSÃO

- › Sem if: cada regra do guardanapo vira uma conta.
- › Ataque, defesa, crítico e vida entram pelo Inspetor (@export).
- › Só está pronto quando passar em todos os casos de teste.

O GUARDANAPO

1. Dano base = ataque – defesa, mas nunca menos que 1.
2. O crítico multiplica o dano base (de 1.0 a 3.0).
3. A vida nunca fica negativa.

OBJETIVO

Uma calculadora de dano que passe nos 4 casos de teste.

Balanceamento de combate

DESAFIOS

- 1 Traduzam cada regra do guardanapo em uma linha de código – sem if. Dica: max e clamp.
- 2 Para cada caso de teste, troquem os 4 valores no Inspetor, rodem e comparem com a tabela.
- 3 Marquem ✓ ou X em cada caso. Falhou? Achem o bug antes de chamar o professor.
- 4 **Bônus:** criem um 5º caso que a dupla vizinha não consiga prever.

CASOS DE TESTE

Ataque	Defesa	Crítico	Vida	Dano	Vida final	Passou?
25	10	2.0	100	30.0	70.0	
5	10	1.0	100	1.0	99.0	
60	10	3.0	100	150.0	0.0	
12	12	1.5	40	1.5	38.5	

FORMATO DA SAÍDA

Dano: 30.0 | Vida restante: 70.0

PENSE E RESPONDA

Qual regra foi mais difícil de escrever sem if? Qual função salvou vocês?

AULA 01 • SEÇÃO 07

Manipulação de tipos

Descobrir, converter e validar tipos sem perder informação pelo caminho.

07

Descobrendo o tipo de um valor

```
var coisa = 3.5

print(typeof(coisa))           # 3 (TYPE_FLOAT)
print(type_string(typeof(coisa))) # float
print(coisa is float)          # true
print(coisa is int)            # false
```

Útil com variáveis dinâmicas (sem tipo), que podem trocar de tipo enquanto o jogo roda.

typeof(x)

Devolve o número do tipo, comparável com TYPE_INT, TYPE_FLOAT, TYPE_STRING...

type_string(n)

Traduz esse número para texto legível: "int", "float", "String".

x is Tipo

Pergunta direta: devolve true ou false.

Convertendo entre tipos

DE → PARA	COMO	RESULTADO	OBSERVAÇÃO
String → int	int("42") · "42".to_int()	42	texto sem número vira 0
String → float	float("3.5") · "3.5".to_float()	3.5	o texto usa ponto, não vírgula
int → String	str(42)	"42"	funciona com qualquer tipo
float → int	int(3.99)	3	corta , não arredonda
int → float	float(7)	7.0	evita a divisão inteira
bool → int	int(true)	1	false vira 0
int → bool	bool(0)	false	só o 0 vira false
vários → String	str("Nível ", 5)	"Nível 5"	junta tudo em um texto

Arredondar ou cortar?

FUNÇÃO	COM 3.7	COM -3.7	DEVOLVE
int(x)	3	-3	int • corta em direção ao zero
round(x)	4.0	-4.0	float • arredonda para o mais próximo
roundi(x)	4	-4	int • arredonda para o mais próximo
floor(x)	3.0	-4.0	float • sempre para baixo
ceil(x)	4.0	-3.0	float • sempre para cima
floori(x) • ceili(x)	3 • 4	-4 • -3	int • para baixo • para cima



Regra de bolso: vai guardar ou mostrar como inteiro? Use as versões terminadas em **i** – roundi, floori, ceili.

A tipagem trabalha por você

CÓDIGO	A GODOT DIZ	POR QUÊ
<pre>var vida: int = 100 vida = "cem"</pre>	ERRO	O editor marca a linha antes de rodar: um String não cabe em um int.
<pre>var coisa = 100 coisa = "cem"</pre>	PERMITIDO	Variável dinâmica aceita qualquer tipo — e esconde erros até o jogo rodar.
<pre>var moedas: int = 10 moedas = 2.9</pre>	AVISO	Converte sozinho e perde a parte decimal: moedas passa a valer 2.
<pre>print("Vida: " + 100)</pre>	ERRO	Texto só se junta com texto. Corrija com "Vida: " + str(100) ou "Vida: %d" % 100 .

Validando texto antes de converter

```
print("42".to_int())           # 42
print("abc".to_int())          # 0 ← cuidado!
print("7 vidas".to_int())      # 7

print("42".is_valid_int())     # true
print("4.5".is_valid_int())    # false
print("4.5".is_valid_float()) # true
```

Todo texto digitado chega como **String**. Antes de fazer contas:

- 1 **Valide** com `is_valid_int()` ou `is_valid_float()`.
- 2 **Converta** com `to_int()` ou `to_float()`.



Atenção: `to_int()` nunca dá erro. Um texto sem números vira 0 — e o bug passa despercebido.

Na próxima aula, com o **if**, vamos reagir quando a entrada for inválida.

ATIVIDADE 7 • BRIEFING

Missão: você é o compilador

Tempo: 20 min

Formato: individual, depois em equipe

Entrega: aula01/atv07_compilador.tscn

Hoje vocês pensam como a Godot. Cada linha de código vale pontos para quem prever exatamente o que vai aparecer na Saída — antes de rodar.

REGRAS DA MISSÃO

- › 3 minutos de previsão individual, em silêncio.
- › Depois, a equipe discute e pode mudar a resposta — com justificativa.
- › 2 pontos por previsão individual certa; 1 ponto pela da equipe.

OBJETIVO

Prever a saída de 8 linhas, explicar cada erro e consertar o código mudando o mínimo possível.

VOCÊ VAI USAR

divisão inteira

int() • roundi() • str()

to_int()

ATIVIDADE 7 • MISSÃO

Você é o compilador

DESAFIOS

- 1 Previsão:** escrevam no caderno a saída de cada linha, sem rodar.
- 2 Correção:** rodem e confirmam. Para cada erro, uma frase: “errei porque...”.
- 3 Conserto:** mudem o mínimo para a linha 1 mostrar 2.5 e a linha 3 mostrar 10.
- 4 Caça ao aviso:** uma linha faz o editor mostrar um aviso amarelo. Qual? Por quê?

```
func _ready() -> void:
    print(10 / 4) # 1
    print(10 / 4.0) # 2
    print(int(9.99)) # 3
    print(roundi(9.5)) # 4
    print(str(3) + str(4)) # 5
    print(int("3") + int("4")) # 6
    print("12 moedas".to_int() * 2) # 7
    print(type_string(typeof(7 / 2.0))) # 8
```

PENSE E RESPONDA Qual armadilha pegou mais gente na turma? Como evitar da próxima vez?

AULA 01 • SEÇÃO 08

Entrada de dados

Como pedir informações ao usuário: Terminal da turma, Inspetor e teclado.

08

Três caminhos para receber dados

Em Python existe `input()`. Na Godot, o painel Saída só mostra texto — ele não recebe digitação. Por isso, usamos três caminhos:

1

Terminal da turma

O programa **pergunta** e espera a resposta, como um `input()`.

Ideal para formulários, cadastros e menus.

CAMINHO PRINCIPAL

2

Inspetor • @export

Você define os valores **antes** de rodar.

Ideal para testar vários valores rapidamente.

3

Teclado • Input

O programa reage a teclas **enquanto** roda.

Ideal para contadores e ações de jogo.

Caminho 1 – Perguntar pelo Terminal

```
um formulário de terminal

extends Node

const ANO_ATUAL := 2026

func _ready() -> void:
    Terminal.escrever("=== CADASTRO DE HERÓI ===")
    var nome: String = await Terminal.perguntar("Nome: ")
    var ano: int = await Terminal.perguntar_int("Ano: ")
    var idade: int = ANO_ATUAL - ano
    Terminal.escrever(nome, " tem ", idade, " anos.")
```

JANELA DO JOGO

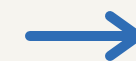
```
=== CADASTRO DE HERÓI ===
Nome: Lia
Ano: 2010
Lia tem 16 anos.
```

💡 **await** = “espere a resposta”. Sem ele, a Godot acusa erro.

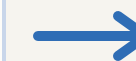
1 • Perguntar
perguntar()



2 • Converter
perguntar_int()



3 • Calcular
ANO_ATUAL - ano



4 • Mostrar
escrever()

As funções do Terminal

FUNÇÃO	DEVOLVE	O QUE FAZ
escrever(valor, valor, ...)	—	Mostra valores, igual ao print()
await perguntar(texto)	String	Espera o usuário digitar e teclar Enter
await perguntar_int(texto)	int	Repete até receber um inteiro válido
await perguntar_float(texto)	float	Repete até receber um número; aceita vírgula
await confirmar(texto)	bool	Pergunta s/n
await menu(titulo, opcoes)	int	Lista opções numeradas e devolve a escolhida
limpar() • pausar() • encerrar()	—	Limpa a tela, espera Enter, finaliza



perguntar_int e **perguntar_float** já validam a entrada. Com **perguntar**, a resposta é texto: converta e valide você mesmo (Seção 07).

Caminho 2 – Inspetor com @export

```
média de três notas
```

```
extends Node
```

```
@export var nota1: float = 7.0
```

```
@export var nota2: float = 8.5
```

```
@export var nota3: float = 6.0
```

```
func _ready() -> void:
```

```
    var media: float = (nota1 + nota2 + nota3) / 3
```

```
    print("Média: %.1f" % media)
```

SAÍDA (OUTPUT)

Média: 7.2

1 Declare as entradas com **@export**



2 Selecione o nó raiz na aba **Cena**



3 Troque os valores no **Inspetor**



4 **Cmd+R** e leia a Saída



Dica: para testar outro caso, basta mudar o Inspetor e rodar de novo. O código não muda.

Caminho 3 – Teclado com Input

```
extends Node

var pulos: int = 0

func _process(_delta: float) -> void:
    if Input.is_action_just_pressed("ui_accept"):
        pulos += 1
        print("Pulou! Total: ", pulos)
```

O **if** será aprofundado na próxima aula. Por enquanto, leia assim: “se a tecla acabou de ser apertada, então...”.

AÇÃO PRONTA	TECLAS
ui_accept	Enter ou Espaço
ui_cancel	Esc
ui_up · ui_down	setas para cima e para baixo
ui_left · ui_right	setas para os lados

just_pressed é verdadeiro só no quadro em que a tecla desce. Para “enquanto segura”, use **is_action_pressed**.

 **Atenção:** clique na janela do jogo. Ela precisa estar em foco para receber as teclas.

ATIVIDADE 8 • BRIEFING

Missão: o mercador de poções

Tempo: 25 min

Formato: em dupla

Entrega: aula01/atv08_mercador.tscn

Um mercador abriu uma loja na vila. O jogador conversa com ele pelo Terminal: diz o nome, pede poções e recebe a conta. A loja abre amanhã — hoje é dia de teste.

REGRAS DA MISSÃO

- › Cada poção custa 15 moedas, sempre (constante).
- › O jogador começa com 100 moedas, ajustáveis no Inspetor.
- › Toda a conversa acontece pelo Terminal. Sem if: problemas viram anotações.

OBJETIVO

Programar a conversa com o mercador e rodar os 3 casos de teste da loja.

VOCÊ VAI USAR

Terminal.perguntar

perguntar_int

const • @export

/ e %

O mercador de poções

CASOS DE TESTE DA LOJA • JOGADOR COM 100 MOEDAS		
Nome	Pede	O mercador deve responder
Lia	3	Lia comprou 3 poções por 45 moedas. Saldo: 55. Ainda compra 3.
Rui	0	Rui comprou 0 poções por 0 moedas. Saldo: 100. Ainda compra 6.
Ana	10	<i>Rodem e anotem o que acontece.</i>

DESAFIOS

- 1

Desenhem no caderno o fluxo da conversa: o que o mercador pergunta, e em que ordem.
- 2

Programem a conversa nesta ordem: perguntar o nome, perguntar a quantidade, calcular e responder.
- 3

Rodem os 3 casos de teste. Qual deles revela um problema?
- 4

Bônus: no fim, o mercador agradece com o nome em maiúsculas: “Obrigado, LIA!”.

PENSE E RESPONDA

O que a loja deveria fazer no caso da Ana? Escrevam a regra em português – na próxima aula, ela vira if.

Resumo da Aula 01

Rodar

```
Cmd+R  cena atual  
Cmd+.  parar  
Cmd+S  salvar
```

Mostrar

```
print("Vida: ", v)  
Terminal.escrever(v)  
print_rich("[b]!")
```

Callbacks

```
_ready()  1 vez  
_process(delta)  
_input(event)
```

Declarar

```
var x: int = 0  
var y := 2.5  
const MAX := 100
```

Pedir dados • await

```
@export var v: int  
Terminal.perguntar()  
Terminal.menu()
```

Tipos

```
bool  int  
float String  
type_string(typeof(x))
```

Operadores

```
+ - * / % **  
== != < > <= >=  
and or not
```

Converter

```
int() float() str()  
"42".to_int()  
roundi(3.7)
```

Armadilhas da aula: print() junta sem espaço • 7 / 2 dá 3 • "Vida: " + 10 é erro • to_int() de texto inválido dá 0 • perguntar() sem await dá erro

AULA 01 CONCLUÍDA

Hora de praticar

Lista de Exercícios 01: cada questão vira uma cena + script na pasta lista01/.

PRÓXIMA AULA [Tema da Aula 02]